Project Proposal AuroraCast
3 Hour Ahead Kp Forecasting from Live Solar-Wind Data

Author: Sven Rudbog (HSLU MLOps, HS 2026) · **Repository (public):** `https://github.com/Nex-cell/auroracast.git`

1 Problem statement

What & for whom: forecast the planetary geomagnetic index Kp (0–9 in thirds, e.g. 4+ = 4.33) for the next 3-hour Kp interval, plus P(Kp ≥ 5), the chance of a G1+ geomagnetic storm.

Users: aurora watchers, photographers and tour operators deciding whether to go out tonight; secondary: HF-radio and GNSS users who are affected by storms.

Horizon & scope: at each interval start `t` (00, 03, …, 21 UTC) the system forecasts Kp for `[t, t+3h)` i.e. 0–3 h ahead using only data available before `t`; this issued forecast is the one that is scored. Hourly runs in between only refresh the displayed forecast with newer solar wind. Scope: the single global Kp index; local visibility (clouds, darkness) is out of scope.

Baseline	Persistence: Kp(next interval) = last Kp *published* before `t`. NOAA SWPC's official Kp forecast is logged as an external reference.
Success criterion	Tiered, relative to persistence on the held-out test set. **Minimum:** lower MAE than persistence and higher storm (Kp ≥ 5) recall than persistence at equal or better precision. **Target:** MAE ≥ 10 % below persistence and storm recall ≥ 0.50 at precision ≥ 0.40. Stretch: ≥ 15 % below persistence. **Live:** rolling 30-day MAE below persistence. Persistence values are measured in the first EDA; targets are estimates and will be re-justified if the baseline proves unusually strong.

2 Originality & motivation

Past projects: HSLU FS26 *Solar Flare Early Warning System* predicts X-ray flux at the Sun 24 h ahead (cause, not impact). KTH ID2223 2022/23 *Aurora Prediction for Kiruna* takes Kp as a user input and does not forecast it. There are no other space weather projects in the HSLU FS26 or KTH 2023–2026 lists.

What's different: Unlike earlier projects that forecast solar flares at the Sun or classify local aurora from a given Kp, AuroraCast forecasts Earth's geomagnetic response (Kp) itself from live in-situ solar wind measured 1.5 million km upstream.

Why this problem, why me: solar cycle 25 is near maximum, so aurora is regularly visible even in Switzerland and many people check Kp. Labels arrive automatically every 3 h, and the declining cycle gives real concept drift to monitor. I bring prior experience with PostgreSQL and Airflow, so I can focus on the ML system rather than learning the infrastructure.

3 Data source & features

Live source	NOAA SWPC official JSON API (no scraping, public domain): `services.swpc.noaa.gov/json/rtsw/rtsw_wind_1m.json` + `rtsw_mag_1m.json`, 1-min resolution, rolling last ~24 h, filtered to the *active* spacecraft. Polled hourly (planned cadence). Fallback: NOAA's propagated solar wind (`products/geospace/propagated-solar-wind.json`).
Data volume	History: NASA OMNIWeb 1-min solar wind since 1995 and the GFZ Kp archive since 1932 → e.g. a 1998–2025 training window ≈ 82 000 labelled intervals (final window chosen after EDA). Growth: +1 440 solar-wind rows and +8 labels per day. The live feed keeps only ~24 h, so my poller stores it from the first run.
Label	Future Kp from GFZ Potsdam (official producer), API `kp.gfz.de/app/json`; nowcast after each interval, definitive values monthly (replace nowcasts). Fallback: SWPC `noaa-planetary-k-index.json`. Kp is measured by 13 ground magnetometers — physically independent of the satellite inputs, not derivable by a rule from the features.
Rare class	Storms (Kp ≥ 5) are only a few % of intervals (exact rate measured in EDA). Planned handling: class weighting in the storm classifier and a decision threshold tuned on validation (resampling as an alternative). Evaluation: PR-AUC, recall, precision and a per-storm-event hit rate — never accuracy.
Features	Initial set (refined after EDA), over e.g. 1 h / 3 h / 6 h windows before `t`: solar-wind speed `V` (mean, max, ΔV shock flag), proton density, dynamic pressure, field `Bt`, `Bz` (mean, min, std, share of minutes with Bz < −5 nT), clock angle, `Ey = −V·Bz`, Newell coupling; lagged Kp (last 1–3 ended intervals), Kp 27 days ago (solar rotation), daily F10.7 flux; sin/cos of UT and day-of-year; data-gap flags.
Leakage / split	Out of time split (fixed principle; initial years: train 1998–2019, validation 2020–2023, test 2024–2025, with purge gaps); live data = true out-of-sample. Leak risks addressed: (1) every feature window ends before `t`; (2) lagged Kp and the persistence baseline use only Kp values already published at `t` (respecting GFZ's publication delay); (3) history uses definitive Kp, live uses nowcast — a small, known train/serve difference, tracked by monitoring; (4) OMNI history is shifted to Earth while live data is at L1 → one shared feature module applies the propagation delay to both.

4 System design

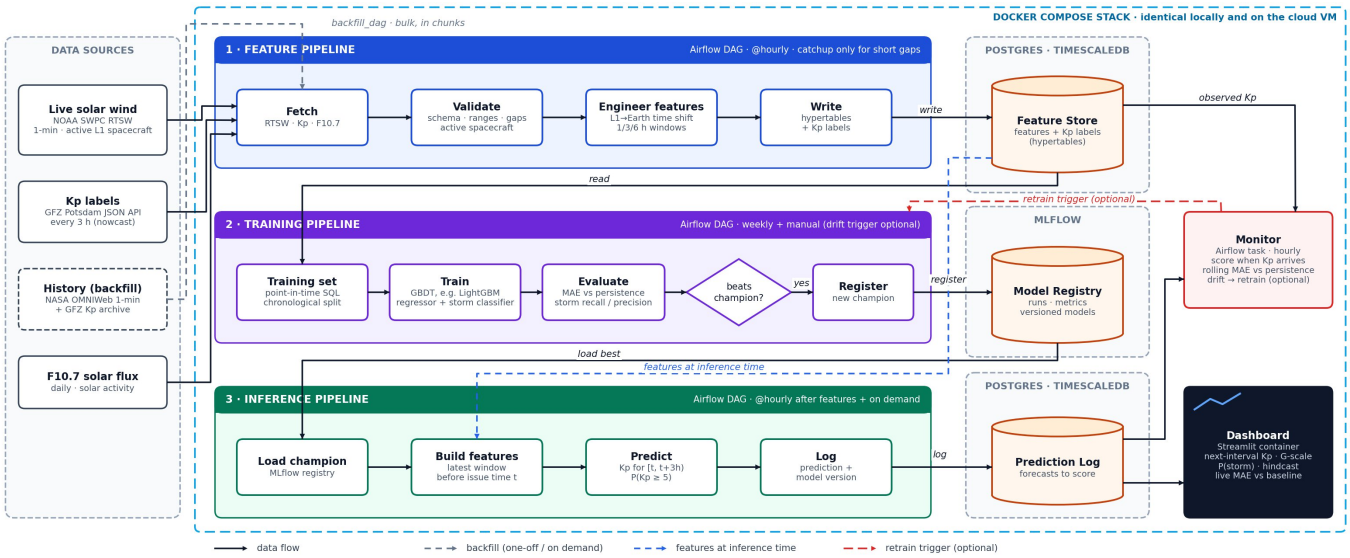


Figure 1 — FTI architecture (draft; editable source in docs/auroracast_fti.drawio).

Pipelines	Feature `feature_dag` (hourly): fetch → validate (schema, ranges, gaps, active spacecraft) → features → feature store; a one-off `backfill_dag` loads history in chunks with the same code. Training `training_dag` (e.g. weekly + manual): point-in-time training set → gradient-boosted trees (initially LightGBM; regressor + storm classifier) → evaluate vs. persistence → register only if better. Inference `inference_dag` (hourly): load champion → predict → prediction log → dashboard. Monitoring (core): each issued forecast is scored once the observed Kp arrives (hindcast + rolling MAE vs. persistence on the dashboard). Schedules and step details are the current plan and may change.
Feature store	PostgreSQL + TimescaleDB time-series hypertables fit 1-min data; plain SQL makes point-in-time queries explicit; I already know Postgres.
Experiment tracking	MLflow logs params/metrics/artifacts and doubles as the model registry (champion alias) in one self-hosted service.
Orchestration	Apache Airflow — schedules and chains the pipelines with retries and a run history; prior experience. The DAGs only wrap plain Python pipeline scripts, so a fallback to GitHub Actions schedules is possible if Airflow becomes a blocker.
Serving	Batch inference into a prediction log, shown in a lightweight dashboard (planned: Streamlit) — enough for a 3-hourly forecast.
Packaging / cloud	Docker Compose runs all services identically locally and on a cloud VM (provider open, e.g. Infomaniak Cloud or Google Cloud, depending on credits); config via `.env`, never committed. Provider-independent by design.
Optional (stretch)	Explicitly *after* the core FTI works: automatic drift-triggered retraining (dashed red in Fig. 1), GitHub Actions CI, NOAA forecast comparison in the UI, automated database backups to object storage, Terraform (IaC).
Repository	Public on GitHub: `https://github.com/Nex-cell/auroracast.git`